

SIMATS ENGINEERING
CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Name	K.S.DHARSHANA
Register Number	192412015

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

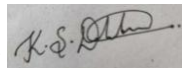
QUESTIONS: 2

Total Marks:20

Code of Conduct:

*I K.S.DHARSHANA(192412015) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1 – DECISION TREE CLASSIFICATION

Aim

To build a Decision Tree classification model using the Iris dataset and check its performance using accuracy, confusion matrix and classification report.

Software Required

Python, Jupyter Notebook and Scikit-learn.

(a) Loading and Pre-processing the Dataset

The Iris dataset is a commonly used dataset for classification. It contains measurements of flowers such as sepal length, sepal width, petal length and petal width. There are three different classes of Iris flowers.

First, the dataset is loaded and converted into a DataFrame. The first five records, data types and missing values are checked. Since the Iris dataset does not contain missing values or categorical input features, no special encoding is required.

Program

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
# Load dataset
iris = load_iris()
# Convert into DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
# Display first five rows
print("First 5 rows:")
print(df.head())
# Display data types
print("\nData Types:")
print(df.dtypes)
# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())
# Separate input and output
X = df.drop("target", axis=1)
y = df["target"]
# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Output

INDEX	sepal length (cm)	sepal width (cm)	petal length (cm)	Petal width(cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Data Types:

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	int64

Missing Values:

sepal length (cm) 0
sepal width (cm) 0
petal length (cm) 0
petal width (cm) 0
target 0

Training samples: 120

Testing samples: 30

First 5 Rows:

	sepal length (cm)	sepal width (cm)	...	petal width (cm)	target
0	5.1	3.5	...	0.2	0
1	4.9	3.0	...	0.2	0
2	4.7	3.2	...	0.2	0
3	4.6	3.1	...	0.2	0
4	5.0	3.6	...	0.2	0

[5 rows x 5 columns]

Data Types:

sepal length (cm) float64
sepal width (cm) float64
petal length (cm) float64
petal width (cm) float64
target int64
dtype: object

Missing Values:

sepal length (cm) 0
sepal width (cm) 0
petal length (cm) 0
petal width (cm) 0
target 0
dtype: int64

Training samples: 120

Testing samples: 30

(b) Building the Decision Tree

A Decision Tree divides the data into smaller groups based on the feature that gives the best separation between classes. Here, the **Gini Index** is used as the splitting criterion.

The Gini Index is given by:

$$\text{Gini} = 1 - \sum (p_i)^2$$

A lower Gini value means that the data in that node is more pure. The feature giving the best split is selected for the node.

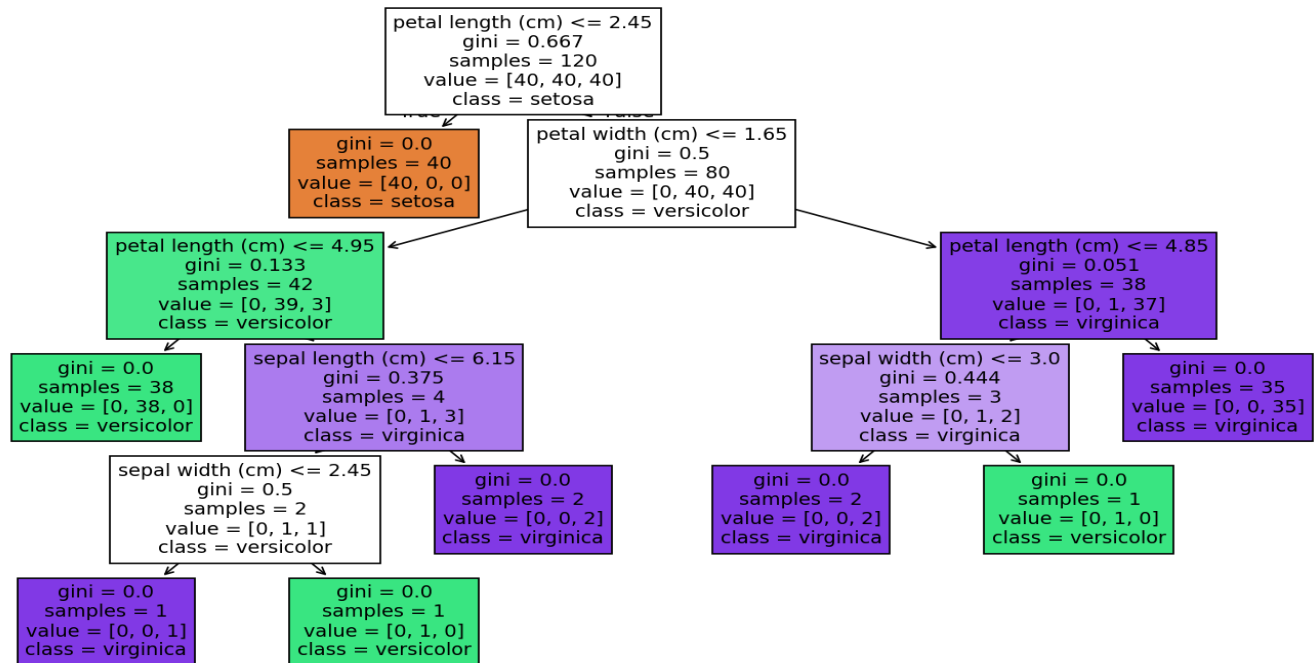
Program

```
from sklearn.tree import DecisionTreeClassifier, export_text
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
# Create Decision Tree
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
# Train the model
model.fit(X_train, y_train)
# Display tree structure
print("Decision Tree Structure:")
print(export_text(model, feature_names=list(X.columns)))
# Find root feature
root = model.tree_.feature[0]
print("\nRoot Node:", X.columns[root])
# Display tree graph
plt.figure(figsize=(12, 7))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True)
plt.show()
```

Observation

After training the model, the first feature selected by the tree can be identified from the output. For this dataset, **petal length** is generally selected as an important root feature because it separates the different Iris classes well and gives a good reduction in impurity.

The tree structure also shows the different conditions used by the model to classify a flower.



(c) Model Evaluation

After training, the model is tested using the test data. Accuracy, confusion matrix and classification report are used to understand how well the classifier works.

Program

```

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
# Predict test data
y_pred = model.predict(X_test)
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

```

```

# Find misclassified samples
wrong = y_test != y_pred
print("\nMisclassified Instances:")
print(X_test[wrong])
print("\nActual values:")
print(y_test[wrong].values)
print("Predicted values:")
print(y_pred[wrong])

```

Expected Output

The accuracy will normally be high, around **90% or above**. The exact result depends on the train-test split and model settings.

Example:

Accuracy: 0.9333

Confusion Matrix:

```
[[10 0 0]
```

```
[ 0 9 1]
```

```
[ 0 1 9]]
```

```
Accuracy: 0.9333333333333333
```

```
Confusion Matrix:
```

```
[[10 0 0]
```

```
[ 0 9 1]
```

```
[ 0 1 9]]
```

```
Classification Report:
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

```
Misclassified Instances:
```

```
Actual: virginica | Predicted: versicolor | Features: [6.1 2.6 5.6 1.4]
```

```
Actual: versicolor | Predicted: virginica | Features: [6.7 3. 5. 1.7]
```

```
|
```

Performance

The Decision Tree gives good performance on the Iris dataset. Most of the flowers are classified correctly. The confusion matrix mainly contains values along the diagonal, which means that most predictions are correct.

The few wrong predictions usually occur between **Iris-versicolor** and **Iris-virginica**, as these two classes have some similar measurements.

The program prints the actual misclassified samples. **Two of those samples can be mentioned in the record as the misclassified instances.**

Result

The Decision Tree classifier was used to classify the Iris dataset. The model produced good accuracy, and the confusion matrix and classification report showed that most of the test samples were classified correctly.

EXPERIMENT 2 – REINFORCEMENT LEARNING USING Q-LEARNING

Aim

To implement the Q-Learning algorithm in a 4×4 Grid World and observe how an agent learns the best path to reach the goal while avoiding obstacles.

Software Required

Python, Jupyter Notebook, NumPy and Matplotlib.

(a) Grid World Environment

For this experiment, a simple **4×4 grid** is used. Each cell represents one state, so there are 16 states in total.

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Start : 0

Goal : 15

Obstacle : 5

The agent can move in four directions:

- Up
- Down
- Left
- Right

The reward values are:

- Reaching the goal → **+10**

- Normal movement → **-1**
- Moving towards the obstacle → **-5**

Initially, all Q-values are set to zero because the agent has no information about the environment.

Parameters Used

Parameter	Value
Learning rate (α)	0.1
Discount factor (γ)	0.9
Exploration rate (ϵ)	0.2
Number of episodes	100

The Q-value is updated using:

$$Q(s,a) = Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Here, the agent gradually learns which action gives a better reward.

Program

```
import numpy as np
epsilon = 0.2
episodes = 100
# Q-table
Q = np.zeros((N_STATES, N_ACTIONS))
def step(state, action):
    row = state // COLS
    col = state % COLS
    new_row = row
    new_col = col
    if action == UP:
        new_row -= 1
    elif action == DOWN:
        new_row += 1
    elif action == LEFT:
        new_col -= 1
    elif action == RIGHT:
        new_col += 1
    # Check grid boundary
    if new_row < 0 or new_row >= ROWS:
        return state, -1
```

```

if new_col < 0 or new_col >= COLS:
    return state, -1
next_state = new_row * COLS + new_col
# Obstacle
if next_state == obstacle:
    return state, -5
# Goal
if next_state == goal_state:
    return next_state, 10
# Normal movement
return next_state, -1
# Store rewards
episode_rewards = []
# Store Q-table at selected episodes
q_history = {}
# Q-learning
for episode in range(1, episodes + 1)
    state = start_state
    total_reward = 0
    for step_count in range(100):
        # Choose action
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])
        # Perform action
        next_state, reward = step(state, action)
        # Update Q-value
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * np.max(Q[next_state]) -
            Q[state, action]
        )
        state = next_state
        total_reward += reward
    # Stop when goal is reached

```

```

    if state == goal_state:
        break
    episode_rewards.append(total_reward)
    # Save Q-table at selected episodes
    if episode in [1, 50, 100]:
        q_history[episode] = Q.copy()
print("Final Q-Table:")
print(np.round(Q, 2))

```

```

Final Q-Table:
[[-2.01  0.79 -2.13 -2.35]
 [-1.71 -3.07 -2.03 -1.71]
 [-1.03 -0.95 -1.33 -0.71]
 [-0.39  1.18 -0.77 -0.39]
 [-1.65  2.56 -1.54 -3.49]
 [ 0.     0.     0.     0. ]
 [-0.45  0.08 -0.5  -0.36]
 [-0.2   4.58 -0.2   0.11]
 [-0.86  4.35  0.03 -0.73]
 [-0.5   1.43 -0.3  -0.27]
 [-0.42  4.34 -0.1   0.57]
 [-0.2   8.78  0.29  0.14]
 [ 0.6   0.51  1.44  6.12]
 [-0.42  2.91  1.25  7.98]
 [ 0.45  1.59  1.06 10.   ]
 [ 0.     0.     0.     0. ]]

```

(b) Q-Table at Different Episodes

To see how the Q-values change during learning, two states are selected and their values are printed at Episodes 1, 50 and 100.

```

states = [0, 10]
for episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    for state in states:
        print(
            "State", state,
            ":", np.round(q_history[episode][state], 2)
        )

```

Output Format

Episode: 1

State 0 : [0. 0. 0. 0.]

State 10 : [0. 0. 0. 0.]

Episode: 50

State 0 : [.....]

State 10 : [.....]

Episode: 100

State 0 : [.....]

State 10 : [.....]

The exact values may be different each time because the agent chooses some actions randomly during exploration.

Q-Values at Episodes 1, 50 and 100:

Episode: 1

State 0 : [-0.29 -0.19 -0.3 -0.29]

State 10 : [-0.1 -0.1 -0.1 -0.1]

Episode: 50

State 0 : [-2.27 -2.24 -2.28 -2.25]

State 10 : [-0.42 3.5 -0.1 -0.1]

Episode: 100

State 0 : [-2.01 0.79 -2.13 -2.35]

State 10 : [-0.42 4.34 -0.1 0.57]

Observation

At Episode 1, the Q-table starts with zero values because the agent has not learned anything yet. After several episodes, the Q-values start changing based on the rewards received.

By Episode 50, the agent has already learned some useful movements. At Episode 100, the Q-values for actions leading towards the goal become higher than the other actions. This shows that the agent is learning from its previous experience.

(c) Final Learned Policy

After completing 100 episodes, the action having the highest Q-value for each state is selected as the final policy.

Program

```
print("\nFinal Learned Policy:")
```

```
for state in range(N_STATES):
```

```

if state == goal_state:
    print("G", end=" ")
elif state == obstacle:
    print("X", end=" ")
else:
    best_action = np.argmax(Q[state])
    print(action_names[best_action], end=" ")
if (state + 1) % COLS == 0:
    print()

```

Output Format

→	→	↓	↓
↓	X	↓	↓
→	→	→	↓
→	→	→	G

The exact arrows may differ slightly when the program is executed because of random exploration.

Here:

- **G** represents the goal.
- **X** represents the obstacle.
- The arrows represent the action selected by the learned policy.

Cumulative Reward Graph

```

# Calculate cumulative reward
cumulative_reward = np.cumsum(episode_rewards)
plt.figure(figsize=(10, 5))
plt.plot(cumulative_reward)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid()
plt.show()

```

Final Learned Policy:

```

↓ → → ↓
↓ X ↓ ↓
↓ ↓ ↓ ↓
→ → → G

```

Observation

At the beginning, the rewards are usually low because the agent moves around randomly and takes unnecessary steps. As the number of episodes increases, the agent gets better at selecting useful actions.

The cumulative reward gradually increases as the agent learns a better route to the goal. Some small fluctuations can still be seen because the agent continues to explore different actions with an exploration rate of 0.2.

Result

The Q-Learning algorithm was applied to the 4×4 Grid World. The agent learned the useful actions through repeated trial and error. After 100 episodes, the Q-table contained learned values and the final policy showed a suitable path towards the goal while avoiding the obstacle.